

Memory Optimisation in High-Level Synthesis under Timing and I/O Constraints

Muhammad Umer Hayat
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
Muhammad-umer.hayat@stud.hshl.de

Abstract—High-Level Synthesis (HLS) allows designers to describe hardware accelerators at the algorithmic level and automatically generate register-transfer level implementations. This improves productivity, but it does not remove the main architectural bottleneck of many signal processing and embedded workloads: memory access. In data-intensive designs, arithmetic units may be available, but operations still stall when several computations request the same memory bank or when external I/O timing does not match the internal schedule. This paper studies memory optimisation within HLS, based mainly on the methodology proposed for synthesis under I/O timing and memory constraints. The central idea is to treat memory mapping and timing behaviour as explicit synthesis constraints instead of handling them after scheduling. The approach uses formal graph models, including the Signal Flow Graph, Algorithmic Constraint Graph, I/O Constraint Graph, Memory Constraint Graph, and Global Constraint Graph, to check feasibility and guide scheduling. Special attention is given to memory conflicts, sequential versus random accesses, and burst-mode scheduling. A 128-point FFT case study is used to discuss the trade-off between memory banks, I/O buses, operators, and latency. The results show that considering memory and I/O constraints together can reduce architecture complexity while still satisfying system-level integration requirements, although this may increase latency. The paper concludes that memory optimisation is not a secondary implementation detail in HLS, but a core co-design problem involving computation, storage, and communication.

Index Terms—High-Level Synthesis, memory optimisation, hardware software codesign, scheduling, memory mapping, FPGA, DSP, I/O timing constraints.

I. INTRODUCTION

High-Level Synthesis has become an important design method in hardware software codesign because it raises the abstraction level of digital hardware design. Instead of writing an accelerator directly in VHDL or Verilog, the designer starts from a behavioural description in C, C++, SystemC, or a similar high-level representation. The HLS tool then performs scheduling, allocation, binding, and RTL generation. This reduces manual RTL effort and allows faster design space exploration.

However, a high-level description does not automatically produce an efficient architecture. The generated hardware still depends strongly on how operations are scheduled and how data is stored and accessed. In many digital signal processing and embedded computing applications, performance is no longer limited only by the number of adders or multipliers. It is

often limited by memory bandwidth, memory bank conflicts, and I/O timing requirements. A design can contain enough arithmetic units and still fail to achieve the required latency because the required operands are not available at the correct clock cycle.

This issue is especially important in hardware software codesign. A dedicated hardware accelerator is rarely isolated. It is normally integrated into a larger system-on-chip or FPGA platform where it communicates with buses, buffers, processors, sensors, and external memory. Therefore, the accelerator must satisfy not only its internal computation schedule, but also external communication constraints such as input data rates, output timing, and bus access patterns.

The paper by Coussy, Corre, Bomel, Senn, and Martin addresses this problem by proposing an HLS methodology that considers both I/O timing and memory constraints during synthesis [1]. Instead of treating memory access as a simple operation after computation scheduling, their approach models memory behaviour formally and integrates it into the scheduling process. This allows the designer to reason about memory placement, sequential and random access costs, memory bank parallelism, and latency trade-offs before RTL generation.

The goal of this paper is to explain and critically discuss memory optimisation in HLS using this methodology as the main reference. The paper is structured as follows. Section II introduces the background of HLS and explains why memory becomes a bottleneck. Section III presents the constraint models used in the design flow. Section IV discusses scheduling under memory constraints. Section V analyses the FFT case study and its design trade-offs. Section VI provides a critical assessment and relates the method to modern HLS design practice. Section VII concludes the paper.

II. BACKGROUND: HLS AND THE MEMORY BOTTLENECK

A. High-Level Synthesis Flow

A typical HLS flow begins with an algorithmic description that specifies what the design should compute, but not exactly how the hardware should be organised. The HLS compiler converts this input into an intermediate representation such as a Control Data Flow Graph or Signal Flow Graph. It then performs scheduling, which assigns operations to control steps or clock cycles. After scheduling, allocation decides how many hardware resources are needed, while binding maps operations

and variables to concrete resources such as multipliers, adders, registers, memory banks, and ports.

This process is powerful because small changes in directives can lead to different hardware architectures. For example, loop pipelining can improve throughput, loop unrolling can increase parallelism, and array partitioning can improve memory bandwidth. In this way, HLS supports design space exploration without forcing the designer to rewrite the complete RTL manually.

B. Why Memory Limits HLS Performance

In software, memory is often treated as a uniform address space. In hardware, memory has physical structure. A memory bank has a limited number of ports, and each port can only serve a limited number of accesses per cycle. If two operations need data from the same single-port memory at the same time, both operations cannot proceed simultaneously. One operation must be delayed, or the memory architecture must be changed.

This makes memory optimisation a central issue in HLS. Consider a simple expression where two multiplications require four variables stored in the same memory bank. Even if two multipliers are available, the hardware may not execute both multiplications in the same cycle because the memory can only provide one operand at a time. The arithmetic parallelism exists, but the memory system cannot feed it. This creates a mismatch between computation parallelism and memory access parallelism.

The usual engineering solution is to add more memory banks or more memory ports. This can increase bandwidth, but it also increases area, routing complexity, power consumption, and sometimes controller complexity. A second solution is to reorganise memory placement so that values accessed in parallel are stored in different banks. A third solution is to schedule operations so that memory accesses follow efficient patterns, for example sequential burst accesses instead of random accesses. Good HLS design needs to balance all of these possibilities.

C. Memory as a Co-Design Problem

Memory optimisation cannot be treated separately from scheduling or I/O timing. If the accelerator receives data from the environment every clock cycle, the internal architecture must be able to consume that data at the required rate. If the output must be produced after a fixed latency, the schedule must satisfy both computation dependencies and memory access limitations. A memory mapping that works for a relaxed latency may fail for a stricter latency. Similarly, an architecture that works with many I/O buses may be unrealistic when the system only provides one or two buses.

This is why the methodology in [1] is relevant for hardware software codesign. It does not ask only how many operators are needed. It asks whether the timing behaviour, memory mapping, and I/O behaviour can all be satisfied together.

III. CONSTRAINT MODELS FOR MEMORY OPTIMISATION

The main contribution of the referenced method is a synthesis flow where algorithmic dependencies, I/O timing, and

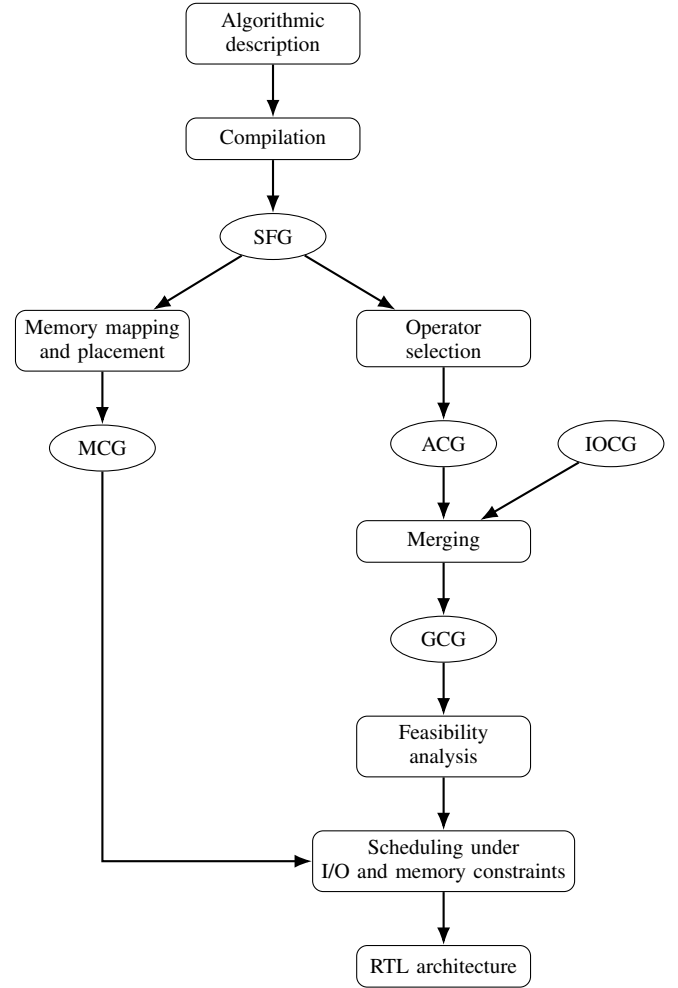


Fig. 1. Simplified HLS design flow with memory and I/O timing constraints. The key point is that the memory model is introduced before scheduling, not after RTL generation.

memory constraints are represented explicitly before scheduling. Fig. 1 shows a simplified version of this flow, redrawn for this paper.

A. Signal Flow Graph

The Signal Flow Graph represents the data dependencies of the algorithm. Operations such as additions, multiplications, and subtractions are represented as nodes, and edges represent the values passed between operations. In DSP algorithms such as FFTs or filters, this graph can be large because many operations are repeated over data streams.

The SFG is important because it exposes which operations depend on previous results and which operations could potentially execute in parallel. However, parallelism in the SFG is only potential parallelism. It can be used only if enough operators and memory access bandwidth are available.

B. Algorithmic Constraint Graph

The Algorithmic Constraint Graph adds timing information to the algorithmic dependencies. Operator latencies are introduced after operator selection. For example, a multiplier

may take two cycles, while an adder or subtractor may take one cycle. These latencies define the earliest and latest times at which operations can be scheduled without violating data dependencies.

In scheduling terminology, this allows the tool to calculate mobility. Mobility is the difference between the latest and earliest possible scheduling time for an operation. An operation with low mobility has less freedom and therefore usually receives higher priority during scheduling. An operation with zero mobility must be scheduled immediately, otherwise the latency constraint cannot be met.

C. I/O Constraint Graph

The I/O Constraint Graph models external communication requirements. It describes data transfer ordering, input and output timing, and rates for ports or buses. This is important because the accelerator may be required to read or write data according to a fixed system schedule.

For example, a component may be constrained to read one input sample per cycle and produce one output sample per cycle after a given latency. These constraints do not come from the algorithm itself. They come from system integration. By representing them explicitly, the HLS tool can check whether the intended component can actually be integrated into the target system.

D. Global Constraint Graph

The Global Constraint Graph is produced by merging the algorithmic and I/O constraint graphs. It captures both internal data dependencies and external communication requirements. This graph is then used for feasibility analysis. If the required output timing cannot be achieved because of input timing, operator latency, or data dependencies, the designer can detect the problem before generating RTL.

E. Memory Constraint Graph

The Memory Constraint Graph is the most relevant model for memory optimisation. It is built from the SFG and the memory mapping file. Its vertices represent data stored in memory, while its edges represent possible consecutive memory accesses. The edge weights distinguish between sequential and random accesses. Sequential accesses correspond to adjacent memory addresses and can support burst-style behaviour. Random accesses correspond to non-adjacent addresses and usually create higher access cost.

The MCG also represents memory conflicts. If two operations require access to the same single-port memory at the same control step, the scheduler cannot schedule both accesses at the same time. The scheduler must either postpone one operation, choose another access order, or report that the current memory mapping cannot satisfy the latency constraint.

This model makes the memory architecture visible during scheduling. That is the main difference from simpler approaches where memory is treated as an abstract storage space and detailed conflicts are discovered late in the design flow.

IV. SCHEDULING UNDER MEMORY CONSTRAINTS

A. Classical List Scheduling

List scheduling is a common scheduling heuristic in HLS. At each control step, the scheduler builds a list of ready operations. An operation is ready when its predecessors have already been scheduled and its operands are available. The operations are then ordered by priority, often based on mobility. Operations with lower mobility are scheduled first because delaying them may violate the latency constraint.

This works well when the main limitation is the number of arithmetic operators. For example, if two multiplications are ready but only one multiplier exists, the scheduler assigns one multiplication and postpones the other. However, this is incomplete when memory constraints are important. An operation may have an available multiplier but still cannot be scheduled because its operands cannot be read from memory in that cycle.

B. Adding Memory Access Criteria

The method in [1] extends scheduling by introducing memory access operators and an accessibility criterion derived from the MCG. A memory bank has as many access operators as it has ports. Therefore, a single-port memory can support only one access at a time, while multiple memory banks or multiple ports can support more parallel access.

At each scheduling step, ready operations are filtered according to memory accessibility. If an operation would create a memory conflict, it is removed from the schedulable list for that cycle. If the operation has positive mobility, it can be postponed. If the operation has zero mobility and a memory conflict still exists, then the current design constraints are infeasible. The designer must change the memory mapping, add memory banks, relax latency, or adjust I/O constraints.

C. Sequential and Random Memory Access

The MCG is also used to prefer efficient access patterns. If two operations have the same scheduling priority and both need the same memory bank, the scheduler can choose the operation that continues a sequential memory access pattern. This is useful because sequential memory accesses can be served more efficiently, especially when burst access is available.

A simple example is shown in Fig. 2. If variables are stored at consecutive addresses, the access order $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4$ is more efficient than repeatedly jumping between non-adjacent addresses. The exact timing depends on the memory technology, but the principle is general: ordered memory access reduces access overhead and simplifies the memory controller.

D. Trade-Off Between Parallelism and Cost

Memory optimisation in HLS is not simply about using the maximum possible number of memory banks. More banks improve parallelism, but they also increase hardware cost and can make memory mapping and routing more complex. Fewer banks reduce area but may force serialization and increase latency.

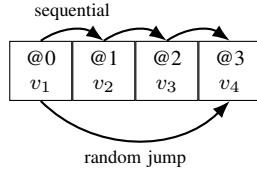


Fig. 2. Memory access intuition. Consecutive addresses favour sequential or burst access, while non-adjacent accesses create a higher access cost and can reduce scheduling efficiency.

TABLE I
FFT SYNTHESIS RESULTS REPORTED IN THE REFERENCE METHODOLOGY

Case	Banks	In	Out	Sub	Add	Mult / Lat.
I/O timing only	8	1	2	10	8	20 / 261
Memory only	2	36	14	10	8	20 / 215
Both constraints	2	1	1	10	8	17 / 343

Latency is measured in clock cycles. “In” and “Out” denote input and output buses.

The practical goal is therefore to find a reasonable balance between memory access parallelism, I/O rates, computation latency, and resource usage. This is exactly the type of trade-off that hardware software codesign is intended to expose. The designer should not optimise the processor, memory, and communication interface independently. The complete architecture must be considered together.

V. FFT CASE STUDY

The original paper evaluates the methodology using a 128-point Fast Fourier Transform. The FFT reads 128 real input samples and produces real and imaginary output values. The Signal Flow Graph contains 8451 vertices and 16897 edges, which makes it large enough to demonstrate non-trivial scheduling and memory behaviour. The experiments use a 200 MHz clock. The multiplier latency is two cycles, while addition and subtraction have a latency of one cycle [1].

The three experiments compare different constraint combinations. Table I summarizes the reported results.

A. Experiment 1: I/O Timing Constraints Only

In the first experiment, the FFT component is constrained mainly by I/O timing. It must read one sample and produce one output sample every cycle, while satisfying a latency of 261 cycles. The resulting architecture uses 20 multipliers, 8 adders, 10 subtractors, and 8 memory banks.

This result shows that strict I/O timing can force memory parallelism. The computation itself can be scheduled, but the data must be supplied fast enough. Therefore, many memory banks are needed to provide enough simultaneous accesses. The advantage is lower latency and a small number of input and output buses. The disadvantage is a more complex memory unit with more banks and more complicated data placement.

B. Experiment 2: Memory Constraints Only

The second experiment uses only memory constraints. The memory mapping is fixed to two banks: real coefficients in one bank and imaginary coefficients in another. With the same number of operators as in the first experiment, the shortest latency is 215 cycles, which is lower than the latency in Experiment 1.

At first this looks better, but the I/O interface becomes unrealistic. The architecture requires 36 input buses and 14 output buses. This means the internal computation can be fast, but only if the surrounding system can provide a very wide communication interface. In many embedded or SoC systems, this is not practical. Therefore, this experiment demonstrates that optimising memory alone can produce a design that is not suitable for system integration.

C. Experiment 3: Both I/O and Memory Constraints

The third experiment combines the memory mapping from Experiment 2 with the I/O rates from Experiment 1. The resulting design uses only two memory banks, one input bus, one output bus, 17 multipliers, 8 adders, and 10 subtractors. The latency increases to 343 cycles.

This is the most balanced solution. It does not achieve the shortest latency, but it satisfies both memory and I/O constraints with a simpler architecture. Compared with Experiment 1, it reduces memory banks from 8 to 2. Compared with Experiment 2, it avoids the unrealistic requirement of 36 input buses and 14 output buses. The cost is increased latency.

This result is important because it shows that the best architecture is not always the fastest architecture. In hardware software codesign, a useful design must be implementable and integrable. A slightly slower design with reasonable memory and I/O structure may be better than a faster design that requires excessive communication resources.

VI. CRITICAL DISCUSSION

A. Strengths of the Methodology

The strongest aspect of the methodology is that it makes constraints explicit. Instead of discovering memory problems after RTL generation, the designer can analyse them during scheduling. This can save design time because infeasible combinations of memory mapping, latency, and I/O timing are detected earlier.

A second strength is that the method connects internal architecture to system integration. The I/O Constraint Graph prevents the designer from producing an accelerator that is internally efficient but externally impractical. This is important for SoC and FPGA platforms where bus widths, data rates, and external memory bandwidth are limited.

A third strength is the use of the Memory Constraint Graph to guide scheduling. The MCG does more than detect conflicts. It also helps choose access orders that favour sequential or burst-mode memory access. This can improve memory efficiency without automatically adding more banks or ports.

B. Limitations

The method also has limitations. First, the designer still plays a major role in memory mapping. If the schedule is infeasible, the tool may indicate that the constraints cannot be satisfied, but the designer must decide whether to change latency, add memory banks, or modify the mapping. This requires experience.

Second, the scheduling algorithm is heuristic. This is understandable because optimal scheduling with realistic resource and memory constraints is computationally difficult for large graphs. However, heuristic scheduling means that the result may not be globally optimal. Another memory mapping or priority rule may produce a better architecture.

Third, the evaluation is based on a DSP workload, specifically an FFT. This is relevant and useful, but it does not automatically prove the same behaviour for all modern workloads. Current HLS applications also include image processing, neural network inference, cryptography, and streaming analytics. These workloads may have different access patterns, reuse opportunities, and memory hierarchy requirements.

C. Relation to Modern HLS Practice

Modern FPGA HLS tools expose memory optimisation through directives such as array partitioning, array reshaping, dataflow regions, stream interfaces, and memory interface pragmas. These directives often give the designer practical control over memory bandwidth, but the underlying problem remains the same. The generated architecture must balance computation parallelism with data movement.

For example, in a convolution accelerator, loop unrolling may create several parallel multiply-accumulate units. If input feature maps and weights are stored in memories that cannot provide enough simultaneous reads, unrolling alone will not deliver the expected speed-up. Array partitioning or buffering is needed to match memory bandwidth to compute parallelism. This is the same principle shown by the FFT case study: arithmetic resources and memory organisation must be co-designed.

The methodology discussed in this paper is therefore still conceptually relevant. Even though tool interfaces have changed, the need for explicit memory reasoning has not disappeared. In fact, it has become more important because modern accelerators often execute data-intensive workloads where memory traffic dominates energy and latency.

D. Possible Future Improvements

A natural improvement would be automated memory mapping. Instead of asking the designer to manually select alternative mappings after infeasibility, the tool could propose candidate mappings based on access patterns. Machine learning or search-based design space exploration could also be used to recommend memory bank allocation, burst-friendly layouts, and latency-resource trade-offs.

Another improvement would be stronger integration with modern memory hierarchies. The original model distinguishes sequential and random access, which is useful, but current

FPGA systems may include BRAM, URAM, caches, DMA engines, AXI buses, and external DDR or HBM memory. A richer model could capture these memory levels and their different bandwidth and latency properties.

Finally, the methodology could be extended toward AI accelerators. Neural network layers have structured reuse patterns, especially in convolution, matrix multiplication, and attention mechanisms. Applying explicit memory constraint models to these workloads could help reduce off-chip transfers and improve energy efficiency.

VII. CONCLUSION

Memory optimisation in High-Level Synthesis is a core hardware software codesign problem. HLS can automatically generate RTL from algorithmic descriptions, but efficient hardware still depends on how data is placed, accessed, and scheduled. If memory constraints are ignored, the resulting architecture may contain enough arithmetic units but still fail to meet timing because operands cannot be delivered at the required rate.

The methodology studied in this paper addresses this issue by integrating I/O timing and memory constraints into the HLS scheduling process. The Signal Flow Graph captures algorithmic dependencies, the ACG and IOCG model computation and communication timing, the GCG supports feasibility analysis, and the MCG represents memory mapping and access conflicts. During scheduling, these models allow the tool to avoid memory conflicts and favour sequential memory access patterns.

The FFT case study shows the central trade-off clearly. Optimising only I/O timing requires many memory banks. Optimising only memory constraints can require an unrealistic number of I/O buses. Considering both constraints together increases latency but produces a more balanced and implementable architecture. This is the main lesson for HLS design: performance, memory, and communication cannot be optimised in isolation. A good accelerator is not only fast, but also feasible, resource-aware, and compatible with the surrounding system.

REFERENCES

- [1] P. Coussy, G. Corre, P. Bomel, E. Senn, and E. Martin, "High-level synthesis under I/O timing and memory constraints," in *Proc. IEEE International Workshop on Rapid System Prototyping*, 2005, pp. 680–683.
- [2] P. Coussy and A. Morawiec, *High-Level Synthesis: From Algorithm to Digital Circuit*. Dordrecht, The Netherlands: Springer, 2008.
- [3] R. Nane, V.-M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi, J. Anderson, and K. Bertels, "A survey and evaluation of FPGA high-level synthesis tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 10, pp. 1591–1604, 2016.
- [4] AMD, *Vitis High-Level Synthesis User Guide*, Advanced Micro Devices, 2023.
- [5] D. D. Gajski, N. D. Dutt, A. C.-H. Wu, and S. Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Boston, MA, USA: Kluwer Academic Publishers, 1992.
- [6] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.